

What is a DBMS?

- Collection of interrelated data — manual or computerized or online
- Set of programs to access the data
- DBMS provides an environment that is both *convenient* and *efficient* to use.

Why do we use DBMS

- To avoid data redundancy and inconsistency
 - ✓ Multiple file formats, duplication of information in different files
- To avoid difficulty in accessing data
 - ✓ Need to write a new program to carry out each new task
- To deal with data isolation — multiple files and formats
- To deal with integrity problems
 - ✓ Integrity constraints (e.g. account balance > 0) become part of program code
 - ✓ Easy to add new constraints or change existing ones

Why do we use DBMS (contd..)

1. Atomicity of updates
 - Failures may leave database in an inconsistent state with partial updates carried out
 - E.g. transfer of funds from one account to another should either complete or not happen at all
2. Concurrent access by multiple users
 - Concurrent accessed needed for performance
 - Uncontrolled concurrent accesses can lead to inconsistencies
 - E.g. two people reading a balance and updating it at the same time
3. Security problems

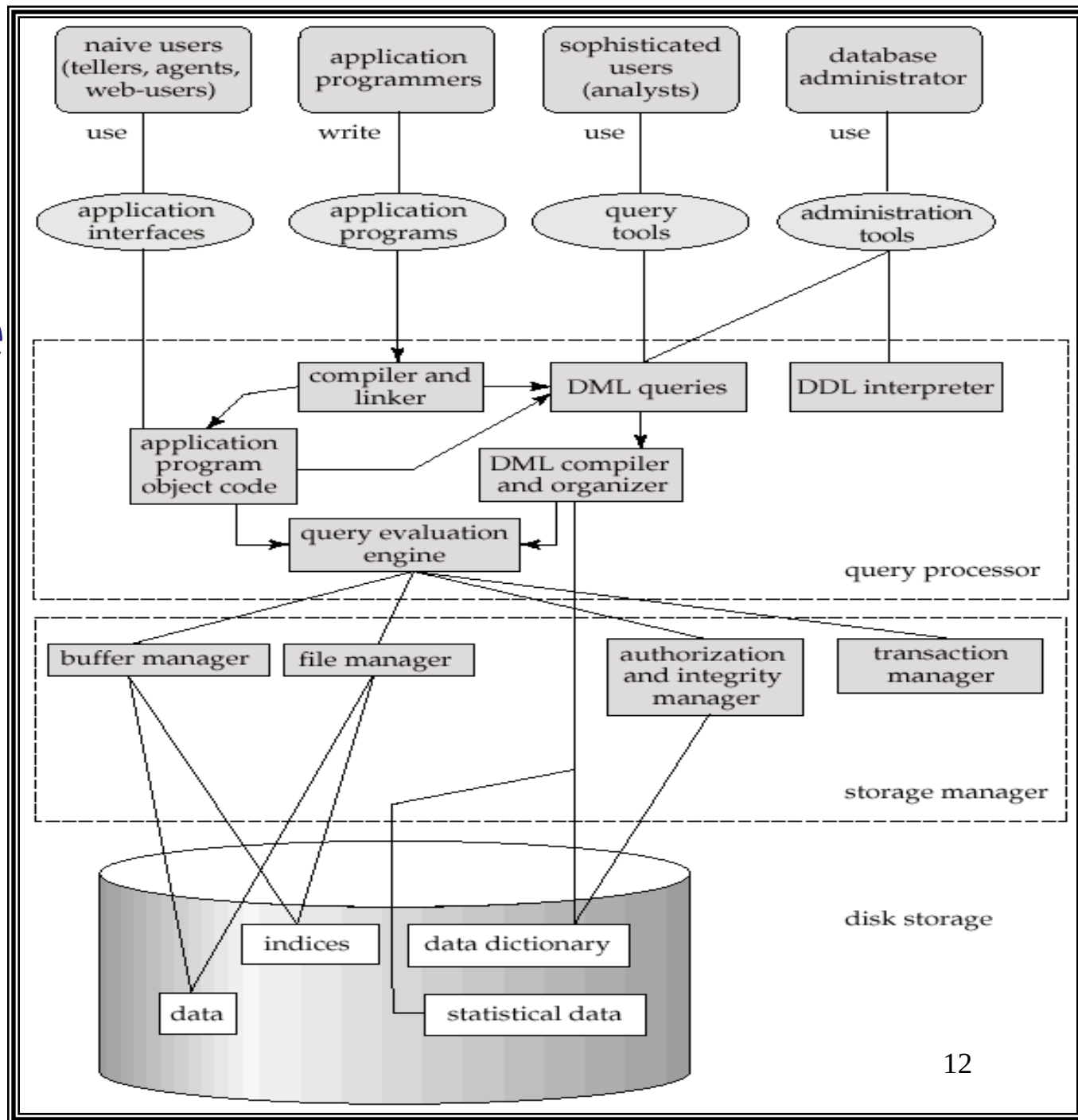
Transaction Management

- A *transaction* is a collection of operations that performs a single logical function in a database application
- Transaction-management component ensures that the database remains in a consistent (correct) state despite system failures (e.g., power failures and operating system crashes) and transaction failures.
- Concurrency-control manager controls the interaction among the concurrent transactions, to ensure the consistency of the database.

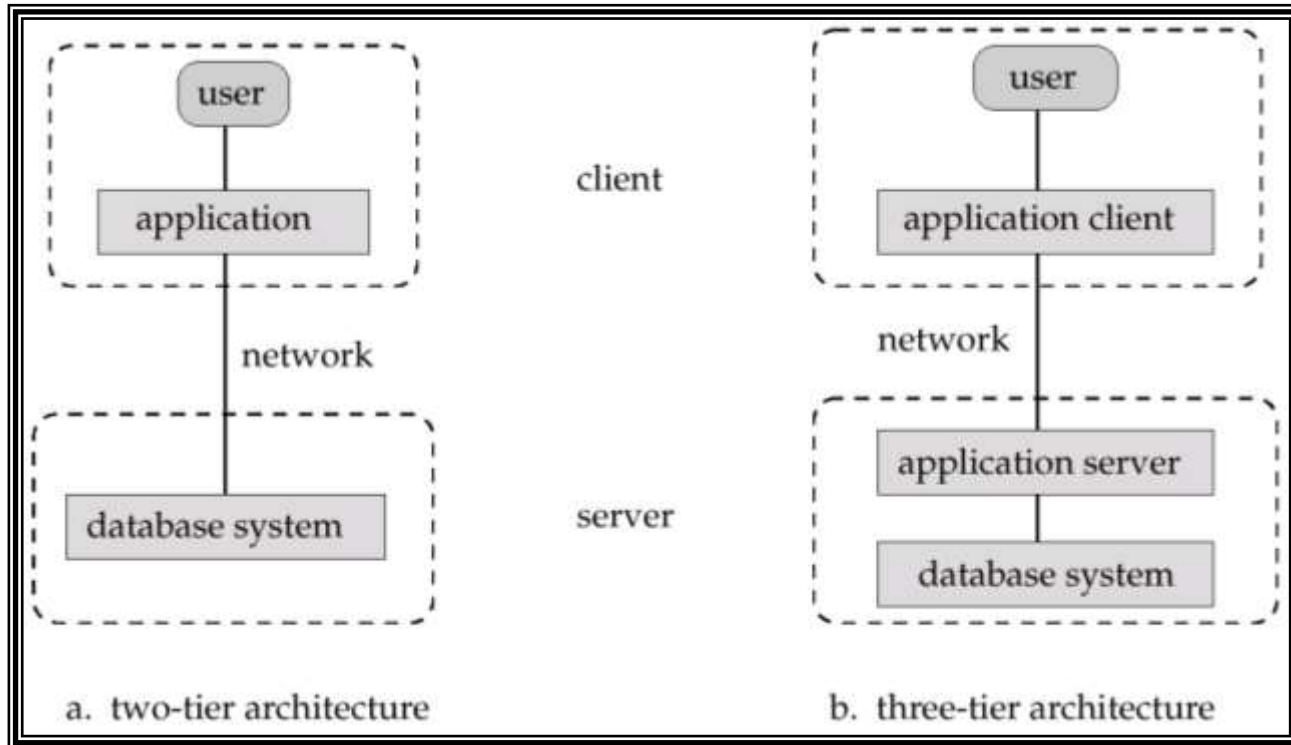
Storage Management

- Storage manager is a program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.
- The storage manager is responsible to the following tasks:
 - interaction with the file manager
 - efficient storing, retrieving and updating of data

Overall System Structure

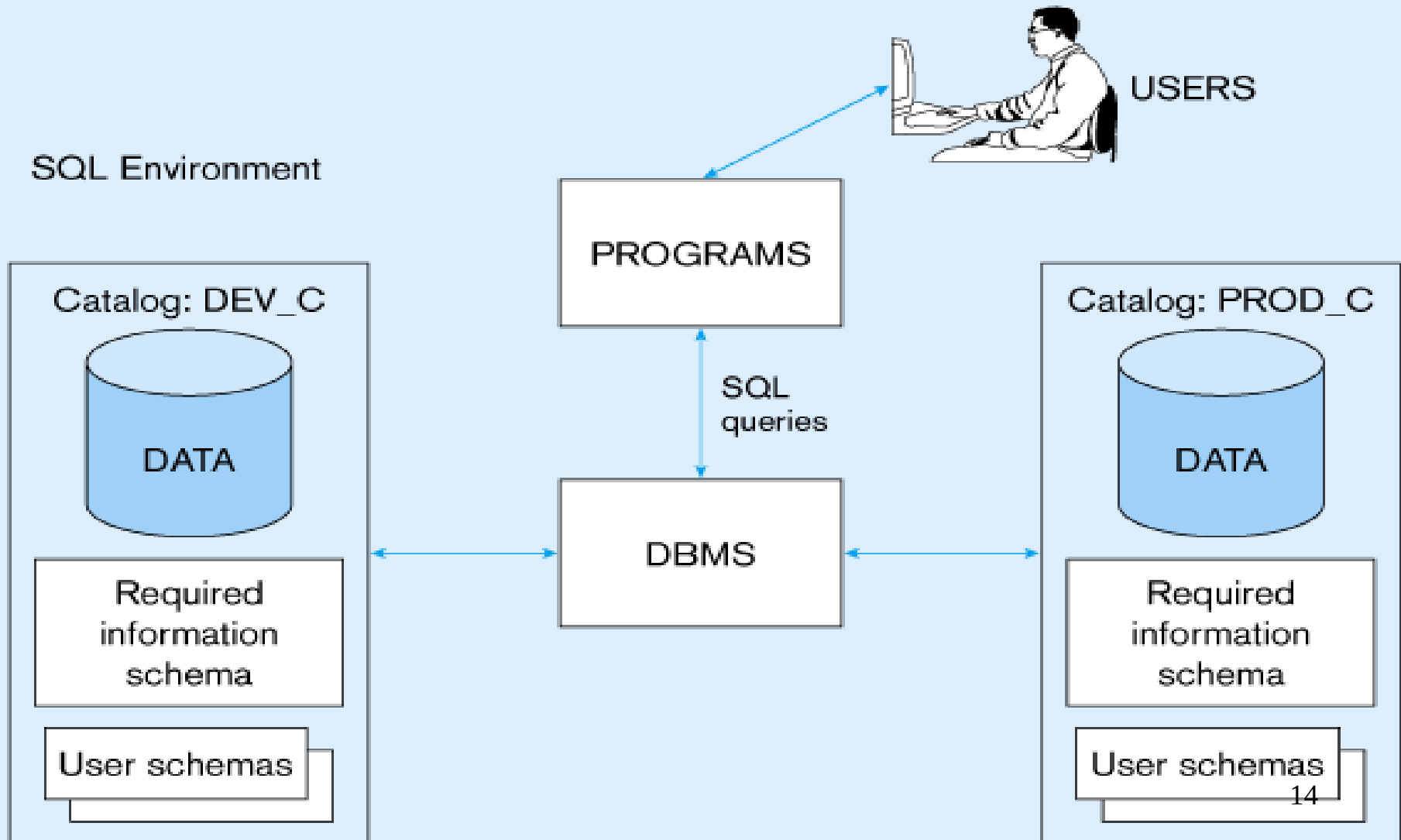


Application Architectures



- **Two-tier architecture:** E.g. client programs using ODBC/JDBC to communicate with a database
- **Three-tier architecture:** E.g. web-based applications, and applications built using “middleware”

DBMS: Allows to Create, Manipulate & Access the Data





How It Works

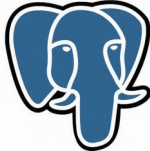


<https://sqlite.org/talks/howitworks-20240624.pdf>



SQLite in a nutshell

- An in-process library → not a server
- One file of ANSI-C code
- Database is a single file
- Full-featured SQL
- Power-safe, serializable transactions
- Fast
- Simple API
- Public domain

	Embedded	Client/Server
Non-SQL	• GDBM • BerkeleyDB • LevelDB • RocksDB • ... and so forth	 MongoDB®  redis  DynamoDB
SQL	 SQLite	 MySQL™  Microsoft SQL Server™  PostgreSQL  ORACLE®

One File Of C-code

sqlite3.c

- 
- 257K lines
 - 163K SLOC¹
 - 9.08MB

- Also: **sqlite3.h**
- 13.4K lines
 - 1.8K SLOC
 - 0.64MB

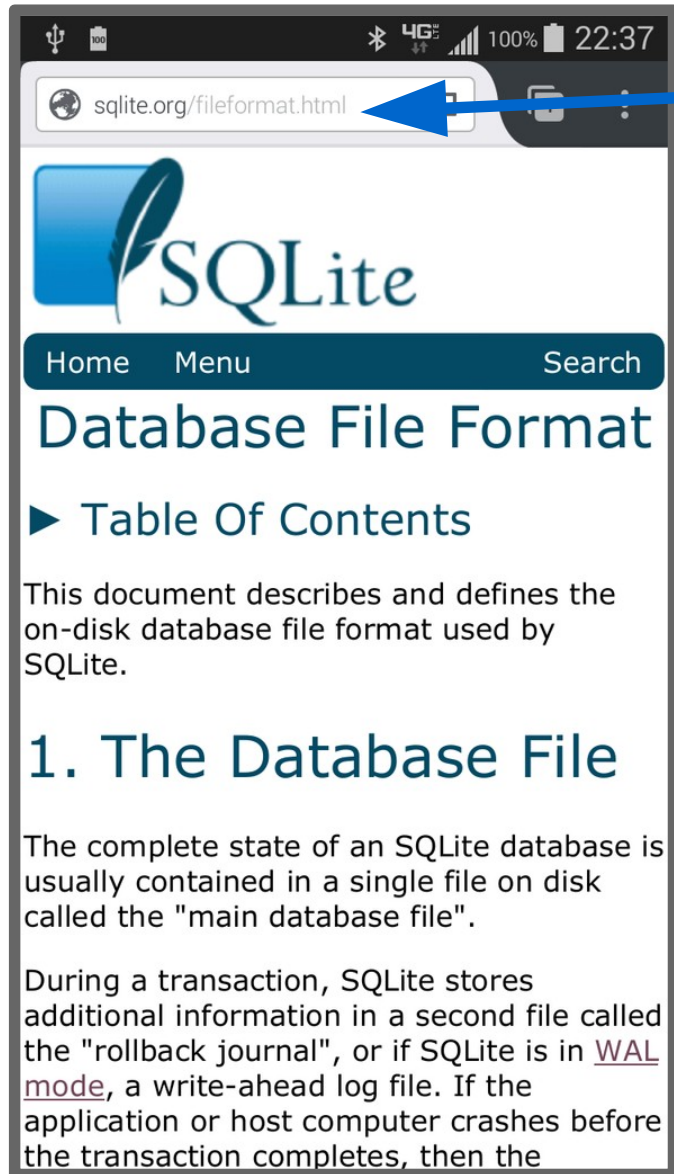
¹SLOC: “Source Lines Of Code” - Lines of code not counting comments and blank lines.

Single File Database

- 1000s of tables, indexes, and views
- Space efficient
- Send a complete database as an email attachment
- Name it whatever you like.
- Application file format

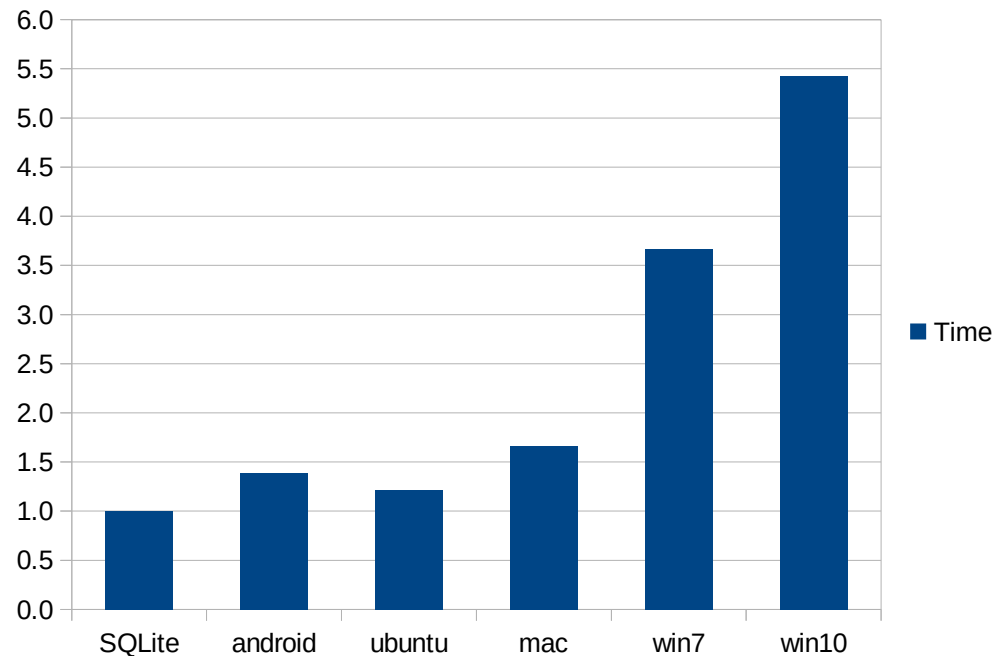


Open File Format



- sqlite.org/fileformat.html
 - ~20 pages
- Cross-platform
 - 32-bit ↔ 64-bit
 - little- ↔ big-endian
- Backwards-compatible
- Space efficient
- Readable by 3rd-party tools
- Supported through 2050
- Recommended by LoC

Faster Than The File System



Time to read 100,000 BLOBs with average size of 10,000 bytes from SQLite versus directly from a file on disk.

<https://sqlite.org/fasterthanfs.html>

“Lite” means “Low Overhead”

not “low capability”

- 1 writer + N concurrent readers
- 1 gigabyte strings and BLOBs
- 281 terabyte databases
- 64-way joins
- 2000 columns per table or index
- No arbitrary limit on the number of tables or indexes or rows in a table

Storage Decision Checklist

Remote Data?

Big Data?

Concurrent Writers?



Otherwise



Storage Decision Checklist **FAIL!**

Remote Data?

Big Data?

Concurrent Writers?



Otherwise

No!

fopen()



SQLite Found In...

- Every Android and iOS phone and device
- Every Mac and Windows10/11 computer
- Every Firefox, Chrome, and Safari browser
- Most desktop and phone software applications
- TVs, smart appliances, automotive “infotainment” systems
- Countless millions of other applications....
- More and more of websites



Implementation Overview

Interface

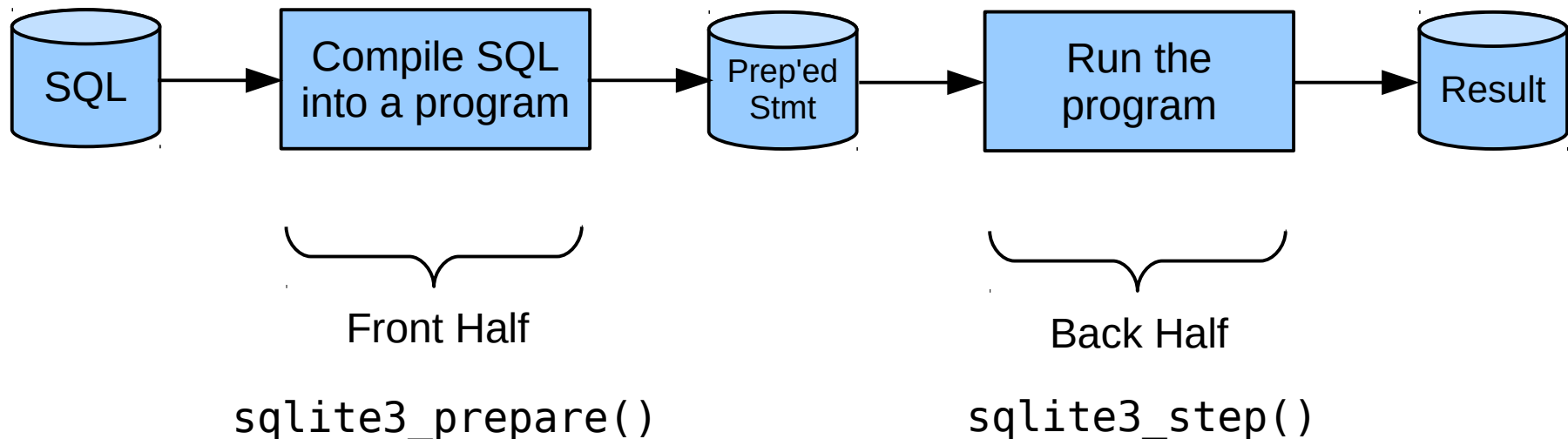
```
int main(int argc, char **argv){
    sqlite3 *db;           /* Database connection */
    sqlite3_stmt *pStmt;  /* One SQL statement */
    int nCol;             /* Number of columns in the result set */

    db = sqlite3_open(argv[1]);
    pStmt = sqlite3_prepare(db, argv[2]);
    nCol = sqlite3_column_count(pStmt);
    while( sqlite3_step(pStmt)==SQLITE_ROW ){
        int i;
        printf("Row:\n");
        for(i=0; i<nCol; i++){
            printf("  %s = %s\n",
                sqlite3_column_name(pStmt, i),
                sqlite3_column_text(pStmt, i)
            );
        }
    }
    sqlite3_finalize(pStmt);
    sqlite3_close(db);
    return 0;
}
```

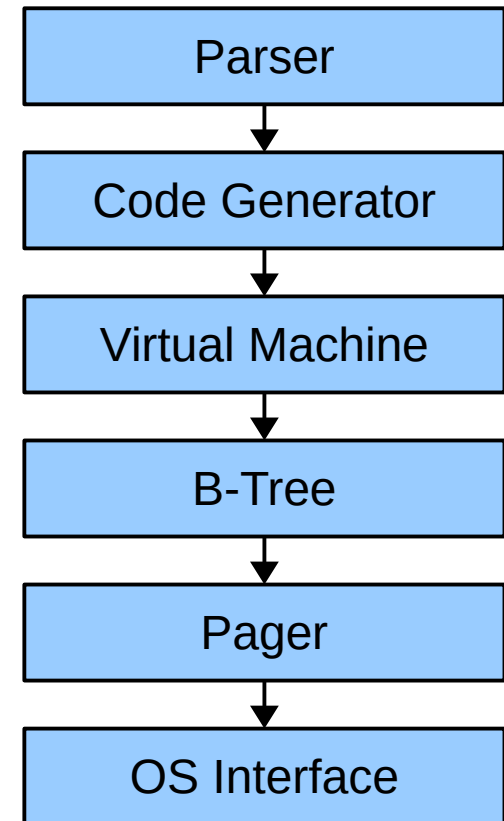
***** Pseudo-code *****

Ins & Outs

- SQLite consists of...
 - Compiler → translates SQL into bytecode
 - Virtual Machine → runs the bytecode

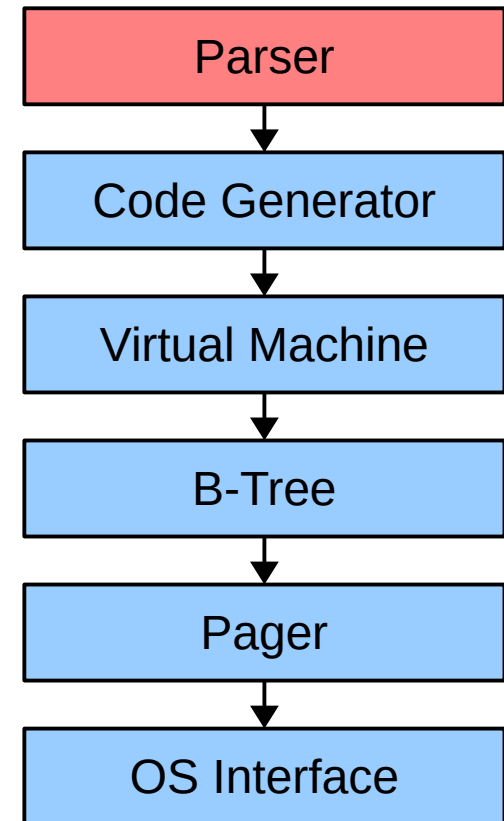


The SQLite Stack



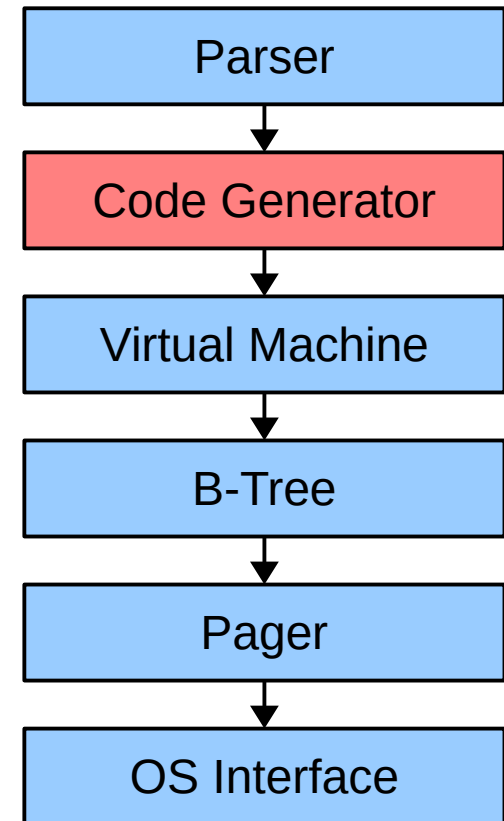
The SQLite Stack

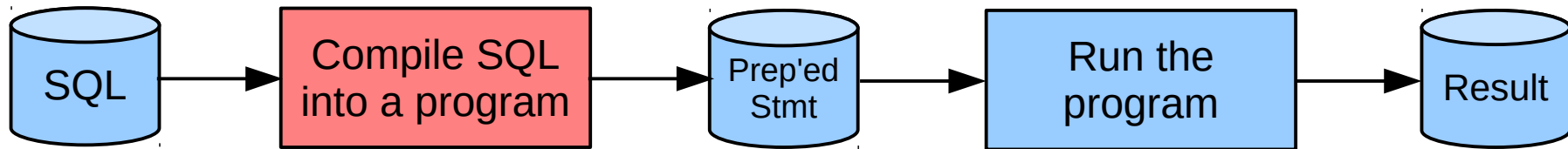
- LALR(1) parser generated by “Lemon”
- Reentrant & threadsafe
- Output: Abstract Syntax Tree (AST)
- Hand-written tokenizer



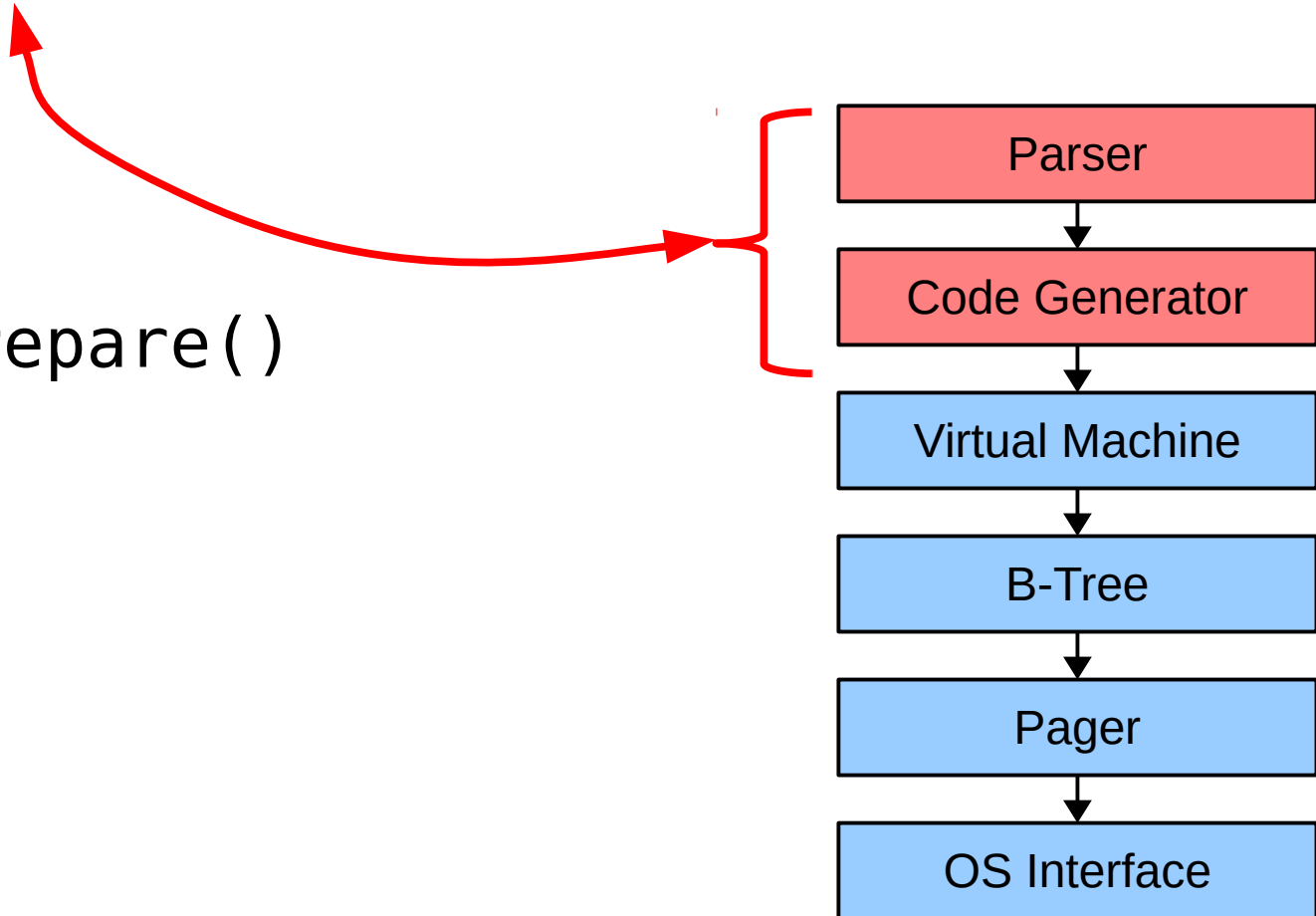
The SQLite Stack

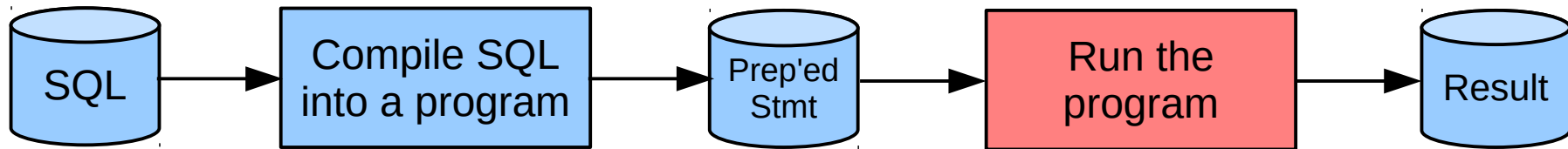
- Semantic analysis
- AST transformations
- Query planning
- Output: “prepared statement” (bytecode)



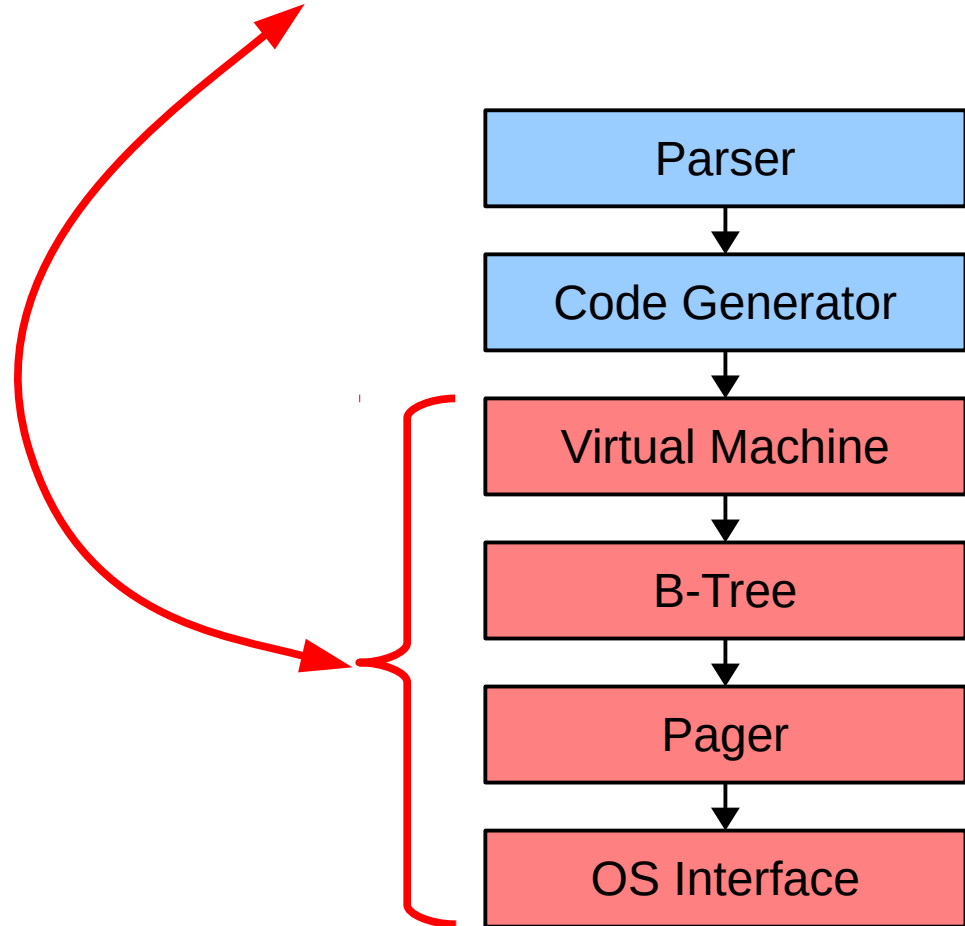


`sqlite3_prepare()`



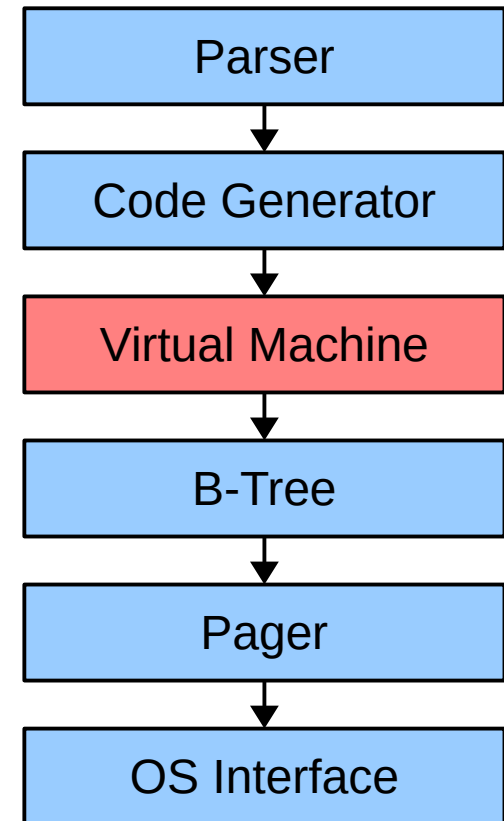


`sqlite3_step()`

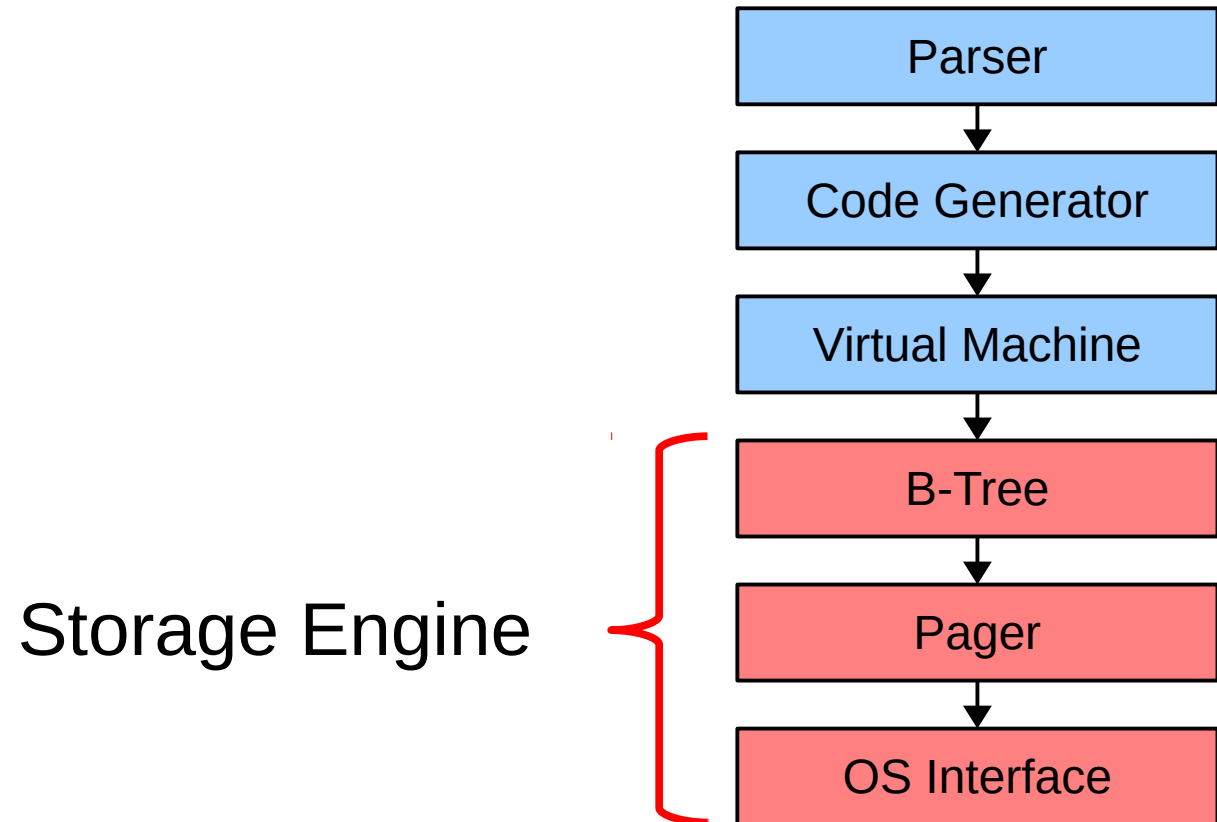


The SQLite Stack

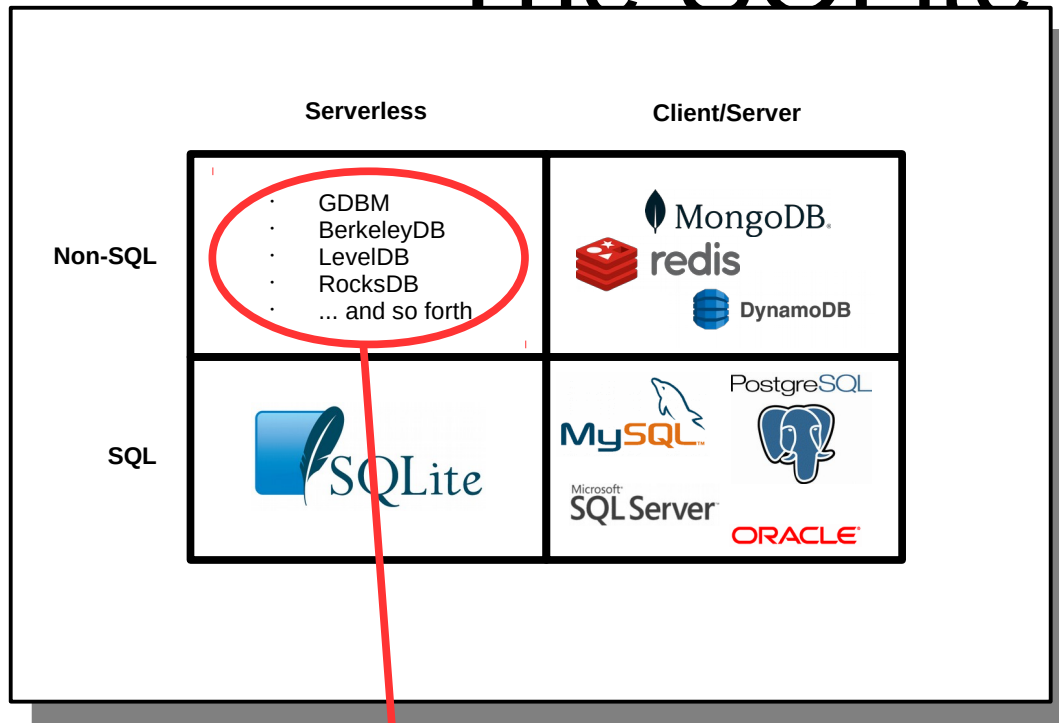
- Bytecode interpreter
- 3-address, 5-operand register machine
- Big switch statement on the opcode and inside a loop.
- Special opcodes designed to help implement SQL statements



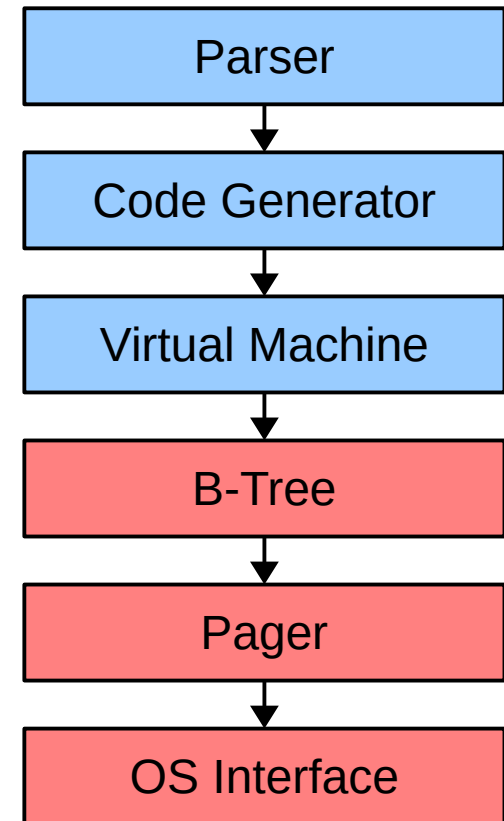
The SQLite Stack



The SQLite Stack



Storage Engine

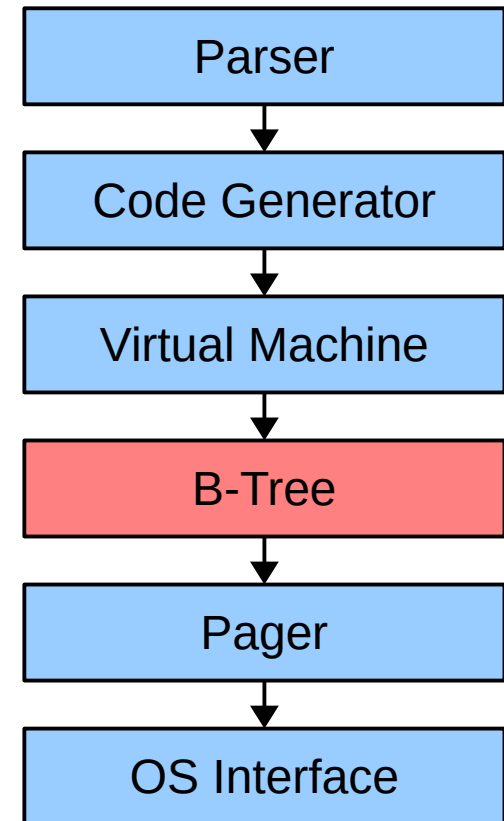


Storage Engine Summary

- Row-store
- Variable-length entries
- Forest of B-trees
 - One B-tree for each table and each index
 - Table key: PRIMARY KEY or ROWID
 - Index key: indexed columns + table key
- Transaction control using rollback-journal or write-ahead log.

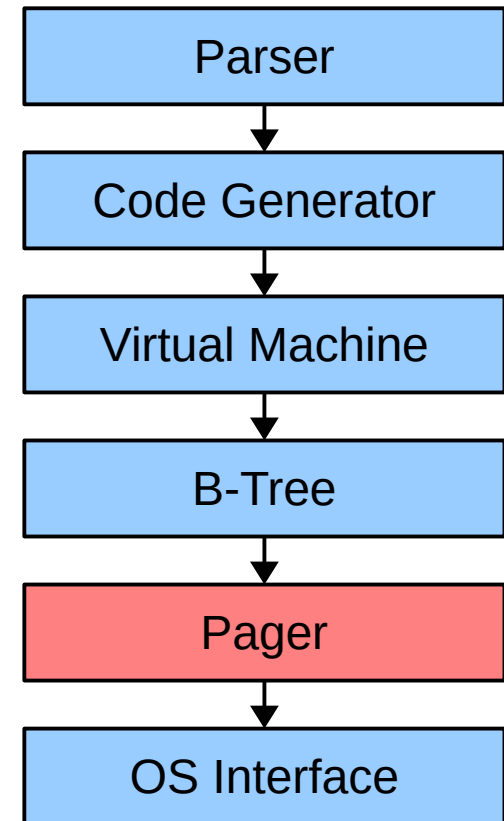
The SQLite Stack

- B-tree and B+trees
- Multiple B-trees per disk file
- Variable-length entries
- Free-page tracking & reuse
- Access via cursor
- Concurrent read/write of the same table using separate cursors



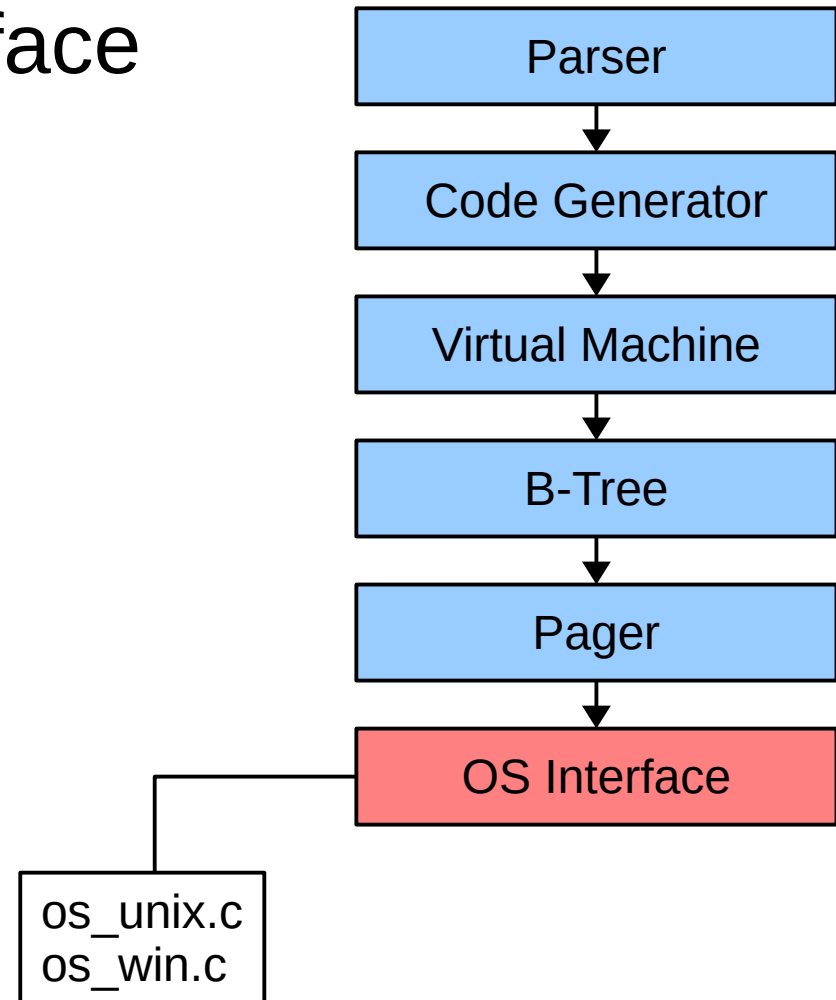
The SQLite Stack

- Atomic commit and rollback
- Uniform size pages numbered from 1
- 512 to 65536 bytes per page
- No interpretation of page content
- Cache
- Concurrency control



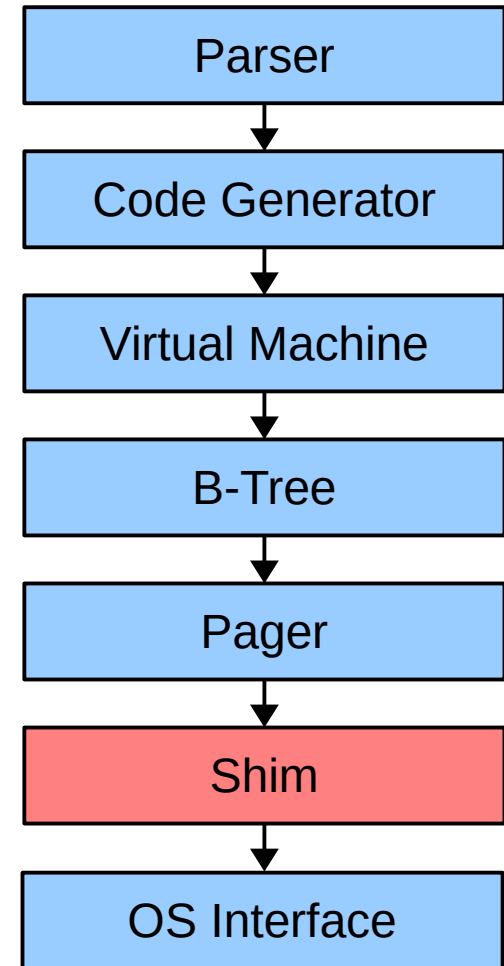
The SQLite Stack

- Platform-specific interface to the OS
- Run-time changeable
- Portability layer
- read()/write() or mmap()
- <https://sqlite.org/vfs.html>
- Direct I/O to hardware: test_onefile.c



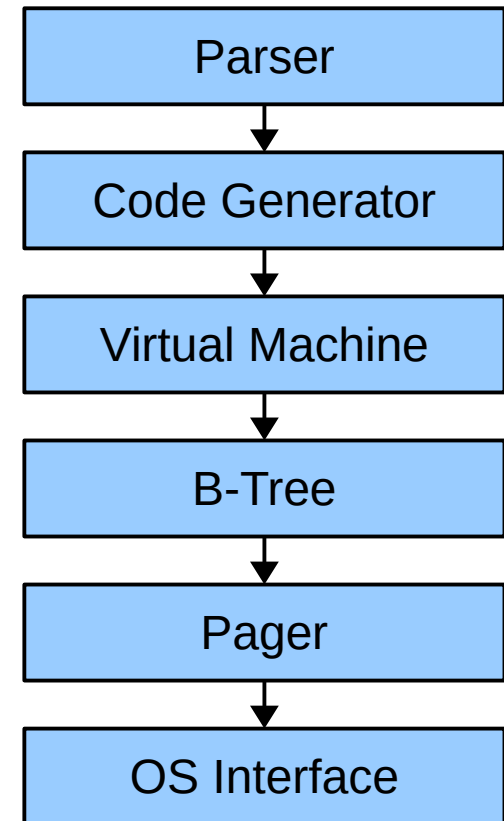
VFS Shims

- Inserted in between Pager and OS Interface
- Encryption
- Compression
- Logging
- Testing & fault injection
- And so forth...



Summary: How SQLite Works

- In-process library. No server.
- One-file database
- Power-safe transactions
- Row-store
- Everything's a b-tree
- Schema stored as a table of CREATE statements
- Bytecode
- Queries planned by solving TSP





Website: <https://sqlite.org/>

Sources: <https://sqlite.org/src/timeline>

Forum: <https://sqlite.org/forum>

